

GrowXth

Plan técnico para convertir el frontend actual en un MVP defendible: búsqueda global con evidencia, ranking explicable y un fallback sólido para la demo.

Decisión en una frase

Next.js sigue siendo el backend del MVP. Exa aporta descubrimiento web en vivo; Apify precarga eventos fuera del request; GitHub aporta actividad técnica agregada; Supabase guarda evidencia y caché. Si una fuente falla, el producto responde parcial o usa datos preparados.

< 8 s

RESPUESTA OBJETIVO

Tres mercados, evidencia y cobertura.

3-4

CONSULTAS EXA

Por búsqueda, ejecutadas en paralelo.

\$0-20

GASTO INICIAL

Suficiente para probar el MVP.

Qué debe funcionar mañana

- **Una consulta real:** el usuario describe producto y objetivo; la app devuelve tres mercados evaluados.
- **Evidencia visible:** cada recomendación enlaza fuentes, indica fecha y nivel de confianza.
- **Datos honestos:** observado, estimado y preparado no se mezclan.
- **Demo resiliente:** sin internet o sin crédito, la misma UI funciona con un dataset local.

Qué queda fuera del MVP

No se construye un scraper universal, no se calcula ROI real sin datos del cliente y no se depende de LinkedIn o de scraping no autorizado de X. La promesa de mañana es descubrir y justificar mercados, eventos y comunidades para developer growth.

Fuente de verdad del producto: el pitch adjunto define el problema y la métrica final; el código actual define qué puede demostrarse mañana.

01 ESTADO ACTUAL

Hay una UI fuerte; falta conectar la inteligencia

El frontend no necesita un rediseño. Compila, pasa lint y ya comunica una plataforma profesional. La brecha principal es que la búsqueda todavía usa escenarios preparados y varios componentes dependen de seis ciudades estáticas.

Área	Ya existe	Brecha para mañana
Frontend	Mapa, animación, ranking, drawer, estados de carga/error y navegación accesible.	El drawer, campaña y nube de señales todavía reciben DemoCity.
Contrato	SearchRequest, SearchResponse, oportunidades, evidencia y cobertura.	No existe todavía POST /api/opportunities/search.
Eventos	Importador de URL pública de Luma con timeout, JSON-LD y cobertura honesta.	Sirve para URLs individuales; no descubre un catálogo global.
Comunidades	Buen schema Python: candidato, revisión, rights basis, evidencia y geografía.	Stores vacíos, sólo SF, sin servicio ni conexión con Next.js.
Datos	Fallback preparado y tres URLs de Luma verificadas.	Señales fijas, sin caché persistente y sin métricas técnicas reales.

Pruebas realizadas sobre la codebase

- `pnpm lint` y `pnpm build`: correctos.
- La vista principal y el flujo de búsqueda se validaron en navegador sin errores de consola.
- Los tres eventos de hackathon se importan: c0mpiled-11 y JacHacks SF con ciudad/coordenadas; YC Startup School con cobertura parcial.
- `sf-community-map/schema.py` compila, pero `normalize_url()` falla con URLs sin esquema pese a que su comentario promete agregar HTTPS.

Dos correcciones de credibilidad

Cambiar “Live signals” por “Prepared data” hasta que la ruta real esté conectada. En fixtures, usar badge “Prepared”; reservar “Observed” para evidencia con URL y fecha verificables.

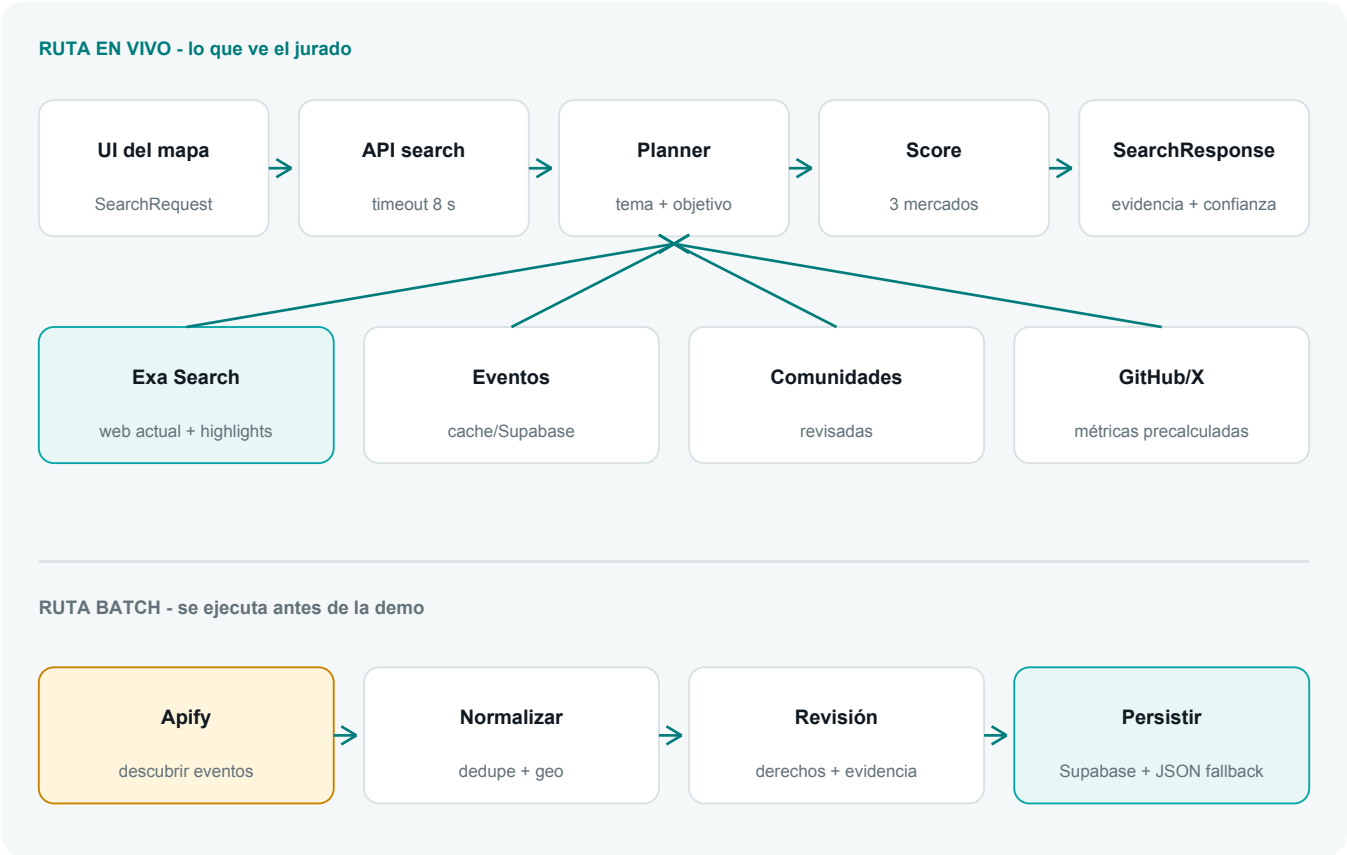
Conclusión de auditoría

La inversión más rentable no es sumar pantallas. Es implementar un puente del contrato existente a una ruta real y eliminar el acoplamiento a DemoCity. Eso permite conservar toda la UI y reemplazar gradualmente los datos preparados por evidencia real.

02 ARQUITECTURA ELEGIDA

Separar la búsqueda en vivo de la recolección

La demo nunca debe esperar a que un actor recorra Luma o a que GitHub procese miles de repositorios. La ruta en vivo combina evidencia rápida con datos ya normalizados; la ruta batch alimenta ese almacén antes del evento.



Capa	Responsabilidad	Regla
Live	Interpretar, recuperar evidencia actual, rankear y responder.	Máximo 8 s; degradación parcial.
Batch	Descubrir, normalizar, deduplicar, geocodificar y revisar.	Nunca bloquea la demo.
Storage	Eventos, comunidades, métricas, evidencia y caché.	Supabase + snapshot JSON local.
UI	Renderizar el contrato, no una fixture concreta.	Live-first, fallback explícito.

Por qué una sola app: para la hackathon, Next.js reduce despliegues, secretos, CORS y puntos de falla. El schema Python de comunidades queda como referencia o herramienta batch; no se invoca desde un request web.

03 ALGORITMO DE BÚSQUEDA

Una respuesta explicable, no un chat que improvisa

La IA puede ayudar a encontrar documentos, pero el ranking debe ser determinista y auditable. Cada razón mostrada en el drawer debe referenciar evidencia concreta.

#	Paso	Implementación mañana
1	Interpretar consulta	Objetivo y presupuesto con parser actual; alias técnicos en tabla local.
2	Elegir candidatos	Seis mercados seed; no afirmar cobertura mundial si sólo se evaluaron seis.
3	Buscar evidencia	3-4 consultas Exa en paralelo: demanda, eventos/comunidades y competencia por ciudad.
4	Unir señales	Eventos y comunidades en storage + métricas GitHub/X precalculadas.
5	Normalizar	URL canónica, dedupe, fecha, ciudad, tipo, rights basis y confianza.
6	Rankear	Score ponderado; razones construidas desde métricas y evidencels.
7	Responder	SearchResponse con cobertura, warnings y timestamps.

Consultas Exa mínimas

Intención	Plantilla
Demanda	{topic} problems OR alternatives {city}
Eventos	{topic} hackathon OR meetup OR developer event {city}
Comunidades	{topic} developer community {city}
Competencia	{category} sponsor OR partner {city}

Política de fallos

Si Exa falla, responder con cache + eventos. Si una fuente individual falla, devolver warning y bajar la confianza. Sólo si la ruta completa falla se usa **demo-adapter**, marcado como fallback.

Caché recomendado

Hash de topic + objective + candidate markets, TTL de 6 horas. Además de bajar costos, permite repetir una demo sin depender de latencia externa.

04 SERVICIOS Y PRESUPUESTO

Pagar sólo donde elimina una limitación real

Servicio	Uso elegido	Cuándo corre	Decisión de gasto
Exa	Descubrimiento web, highlights y evidencia reciente.	En vivo, 3-4 queries.	Usar créditos incluidos; cap de \$10-20.
Apify	Precargar eventos públicos de Luma por ciudad/categoría.	Batch, antes de la demo.	Free \$5. Comprar Starter sólo tras smoke test.
GitHub	Actividad técnica agregada por tema y región.	Batch.	API oficial gratuita con token.
Supabase	Storage, evidencia, caché y timestamps.	Lectura live / escritura batch.	Free tier.
X API	Counts-first para volumen relativo; posts sólo si hacen falta.	Batch opcional.	\$5-10 opcionales; no actor no oficial.
OpenAI/LLM	Opcional: alias, clasificación o resumen estructurado.	Después del retrieval.	No bloquear el MVP por esto.

Costo aproximado por búsqueda

Con precios públicos actuales de Exa, cuatro búsquedas normales cuestan aproximadamente **\$0.028**; recuperar 20 páginas con un tipo de contenido agrega aproximadamente **\$0.020**. Una consulta completa queda cerca de **\$0.05** antes de caché. Es suficiente para una hackathon.

\$0.05

EXA / SEARCH

Estimación conservadora sin caché.

\$5

APIFY FREE

Crédito para validar el actor.

\$0

DB + GITHUB

Free tiers para el MVP.

Reglas de compra

- Ejecutar un smoke test de 20 eventos del actor Luma. Verificar campos, duplicados, latencia y costo real.
- No comprar Apify Starter de \$29 hasta que el actor supere esa prueba.
- Configurar límites de gasto y nunca exponer tokens al cliente.
- Aplicar luego al grant de Exa para startups/educación; no es requisito para mañana.

Decisión sobre X y LinkedIn

X prohíbe scraping/crawling sin consentimiento previo en sus términos actuales. Usar la API oficial o dejar la integración fuera. LinkedIn tampoco debe ser una dependencia del MVP.

05 DATOS, SCORE Y CONFIANZA

Sólo puntuar lo que puede justificarse

El pitch apunta a costo por desarrollador activado y retenido. Esa será la métrica final, pero no puede estimarse honestamente mañana sin datos de campañas y activación del cliente. El MVP debe rankear oportunidad, no inventar ROI.



Dimensión	Evidencia aceptable	Qué no hacer
Demand	Documentos recientes, discusiones, trends/counts agregados.	Contar resultados de búsqueda como demanda absoluta.
Developer fit	GitHub agregado, lenguajes, repos y comunidades técnicas.	Perfilar personas o inferir ubicación como certeza.
Readiness	Eventos futuros, cadencia, tamaño y organizadores.	Inventar premios, sponsors o asistencia.
Gap	Presencia observada de competidores/sponsors.	Interpretar ausencia web como ausencia total.
Confidence	Cobertura, frescura, concordancia y geo confidence.	Mostrar "100% global" por evaluar seis ciudades.

Contrato de evidencia mínimo

`id · source · url · title · observedAt · location · confidence · rightsBasis · kind`

Toda razón del ranking contiene `evidenceIds`. El drawer resuelve esos IDs y muestra links. Los campos inferidos llevan confianza; los campos preparados llevan `status prepared`.

Regla de oro

Si el dato de costo no existe, mostrar "No disponible" o "Estimado" y excluirlo del score. La confianza del producto vale más que una métrica llamativa.

06 ESTRUCTURA DEL PROYECTO

Ordenar por responsabilidad, sin crear otro backend

Cambio	Acción concreta
Contrato	Mover gradualmente <code>lib/api/types.ts</code> a <code>lib/contracts/atlas.ts</code> ; compartirlo entre UI y route.
Cliente	Agregar <code>searchOpportunities()</code> live-first en <code>atlas-client.ts</code> ; fallback al demo adapter.
UI	<code>selectedCityId: string</code> ; drawer, campaña y signal cloud reciben <code>Opportunity</code> .
Comunidades	Exportar registros revisados del schema Python a JSON seed. No ejecutar Python desde Next.js.
Repositorios	Interfaz única con implementaciones Supabase y fixture; el pipeline no conoce el storage concreto.

Estructura objetivo

```
interactive-world-map/  
  app/api/  
    opportunities/search/route.ts  
    events/ingest/route.ts  
  lib/  
    contracts/atlas.ts  
    client/atlas-client.ts  
    server/  
      pipeline/search-opportunities.ts  
      query/interpret-query.ts  
      connectors/{exa,github,apify,x-counts}.ts  
      normalize/{event,community,evidence,location}.ts  
      scoring/opportunity-score.ts  
      repositories/{repository,supabase,fixture}.ts  
      cache/search-cache.ts  
    demo/fixtures.ts  
  data/seed/{cities,events,communities,github-signals}.json  
  scripts/{ingest-luma,ingest-github,review-communities}.ts  
  tests/{contracts,scoring,api}/
```

Variables de entorno

EXA_API_KEY · APIFY_TOKEN · GITHUB_TOKEN · SUPABASE_URL · SUPABASE_SERVICE_ROLE_KEY · X_BEARER_TOKEN(optional)

Compatibilidad con el trabajo de comunidades

Conservar los campos valiosos del schema actual: `candidate` vs `community`, `manualReviewRequired`, `rightsBasis`, `evidenceUrl`, `canonicalUrl` y `geo confidence`. Portar sólo el DTO de lectura al frontend.

07 PLAN DE IMPLEMENTACIÓN

Orden de trabajo para llegar a una demo completa

Bloque	Backend / datos	Frontend / integración	Hecho cuando...
A · 90 min	Ruta search, Exa connector, timeout y schema validation.	Cliente live-first con fallback.	Una query devuelve SearchResponse.
B · 90 min	Seeds: 6 ciudades, 3 eventos verificados, 10-20 comunidades.	Rail y markers usan oportunidades reales.	Los tres resultados abren sin DemoCity.
C · 120 min	Normalizer, dedupe, confidence y evidence IDs.	Drawer, campaign y signal cloud desde Opportunity.	Cada razón enlaza evidencia.
D · 60 min	Smoke test Apify + snapshot; batch GitHub básico.	Badges Observed/Estimated/Prepared.	Una fuente real alimenta el mapa.
E · 60 min	Cache, logs, env guard y respuesta parcial.	Copy honesto y estados de error.	Demo funciona sin una API externa.
F · 60 min	Tests de contract, score y Luma parser.	Build, lint, teclado, responsive y demo run.	Checklist de aceptación completo.

P0 - obligatorio

- Ruta real, Exa connector, storage adapter y fallback.
- Eliminar dependencia de DemoCity en la ruta principal.
- Evidencia, cobertura y labels honestos.
- Cache + timeout + variables de entorno protegidas.

P1 - si P0 ya está estable

- Apify batch con resultados normalizados.
- GitHub agregado y X counts oficial opcional.
- Supabase; conservar siempre snapshot JSON.

No hacer antes de la demo

No migrar todo el frontend, no crear microservicios, no entrenar modelos, no construir un crawler propio y no sumar nuevas categorías de consumo. Cada una aumenta superficie de fallo sin mejorar la historia central.

08 ACEPTACIÓN Y DEMO

La app está lista cuando puede defender cada resultado

Criterio	Aceptación
Search	Consulta arbitraria devuelve 3 mercados en 8 s o menos; si hay fallos, devuelve parcial.
Evidencia	Cada razón tiene fuente clickeable, fecha, tipo y confianza.
Cobertura	La UI dice cuántos mercados/fuentes evaluó; no afirma cobertura global falsa.
Métricas	No hay ROI, costo o sponsor inventado; lo estimado está marcado.
Resiliencia	Fallback local completo y caché disponible sin Exa/Apify.
Calidad	Build y lint correctos, sin errores de consola; flujo usable con teclado.

Guion de 75 segundos

Tiempo	Acción	Mensaje
0-15 s	Escribir producto, objetivo y presupuesto.	"Las empresas no saben dónde su próximo dólar de developer growth rendirá más."
15-35 s	Mapa anima y rankea tres mercados.	"Combinamos demanda pública, actividad técnica, eventos y comunidades."
35-60 s	Abrir #1 y recorrer evidencia.	"El score es explicable; cada señal tiene fuente y confianza."
60-75 s	Mostrar campaña sugerida y cobertura.	"Hoy elegimos mejor; mañana medimos activación y retención."

Definición final del MVP

GrowXth es un motor de developer growth intelligence. Para una empresa y un objetivo, identifica mercados, eventos y comunidades prometedoras y explica la recomendación con evidencia verificable.

Después de la hackathon

Conectar datos privados de activación/retención, reemplazar seeds por pipelines programados, ampliar ciudades e idiomas y aprender del resultado de campañas. Ese historial - no el scraping aislado - es el moat.

09 FUENTES Y TRAZABILIDAD

Referencias usadas para la decisión

Precios y restricciones consultados el 23 de julio de 2026. Antes de un lanzamiento comercial, revisar nuevamente precios, términos y licencias de cada proveedor.

Fuente	URL
[1] Exa API pricing	https://exa.ai/pricing?tab=api
[2] Exa Search API	https://exa.ai/docs/reference/search
[3] Exa Contents API	https://exa.ai/docs/reference/contents-retrieval
[4] Apify pricing	https://apify.com/pricing
[5] Apify Actor API	https://docs.apify.com/academy/api/run-actor-and-retrieve-data-via-api
[6] Luma Scraper actor	https://apify.com/solidcode/luma-scraper
[7] X API pricing	https://docs.x.com/x-api/getting-started/pricing
[8] X Terms of Service	https://x.com/en/tos
[9] GitHub REST rate limits	https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api
[10] GitHub Innovation Graph methodology	https://innovationgraph.github.com/methodology
[11] Supabase pricing	https://supabase.com/pricing
[12] Luma API	https://help.luma.com/p/luma-api
[13] Luma Terms	https://luma.com/terms

Archivos auditados con mayor impacto

- `interactive-world-map/lib/api/types.ts`
- `interactive-world-map/lib/api/demo-adapter.ts`
- `interactive-world-map/lib/api/atlas-client.ts`
- `interactive-world-map/app/api/events/ingest/route.ts`
- `interactive-world-map/lib/api/luma.ts`
- `interactive-world-map/components/atlas/atlas-shell.tsx`
- `sf-community-map/schema.py`

Documento generado a partir del pitch final, la codebase actual, pruebas locales y documentación oficial. La arquitectura prioriza una demo profesional y una migración limpia hacia datos reales.